

Kruskal's algorithm $\Rightarrow$ To find the (MST)

Disjoint Set (Find Parent)

Step 1: Sort the edges in ascending order

MST: $1+2+3+4 = 10$

MST Prequisite: $O(4\alpha)$

N nodes $\rightarrow N-1$

Find the ultimate parent $\rightarrow$ Union by rank

If they don't belong to the same component, it will be part of MST.

(Joseph Kruskal) Mathematician

WT: 4, 1, 1, 2, 2, 3, 3, 4, 3, 6, 4, 5

$S1 \rightarrow (1), 1, (4)$

$S2 \rightarrow (2), 1, (2)$

$S3 \rightarrow (3), 2, (3)$

$S4 \rightarrow (4), 1, (5)$

$S5 \rightarrow (7), 2, (6)$

$\times 3, 2, (4)$

$\times 5, 3, 4$

$\times 8, 3, 6$

$\times 9, 4, 5$

$= 17$

Dynamic Programming

"Those who forget the past, are forced to repeat it."

$\Rightarrow$ It is the process of solving smaller overlapping subproblems to solve a bigger problem.

- Recursion $\{ \text{top-down} \}$
- Memorisation $\{ \text{bottom-up} \}$
- Tabulation
- Space Optimization (optional)

(1D & 2D) (DP)

[1D $\rightarrow$ DP] Fibonacci Sequence: $\rightarrow$

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

0 1 2 3 4 5 6 7 8 ... n

$f(0) = 0$

$f(1) = 1$

$f(2) = f(1) + f(0)$

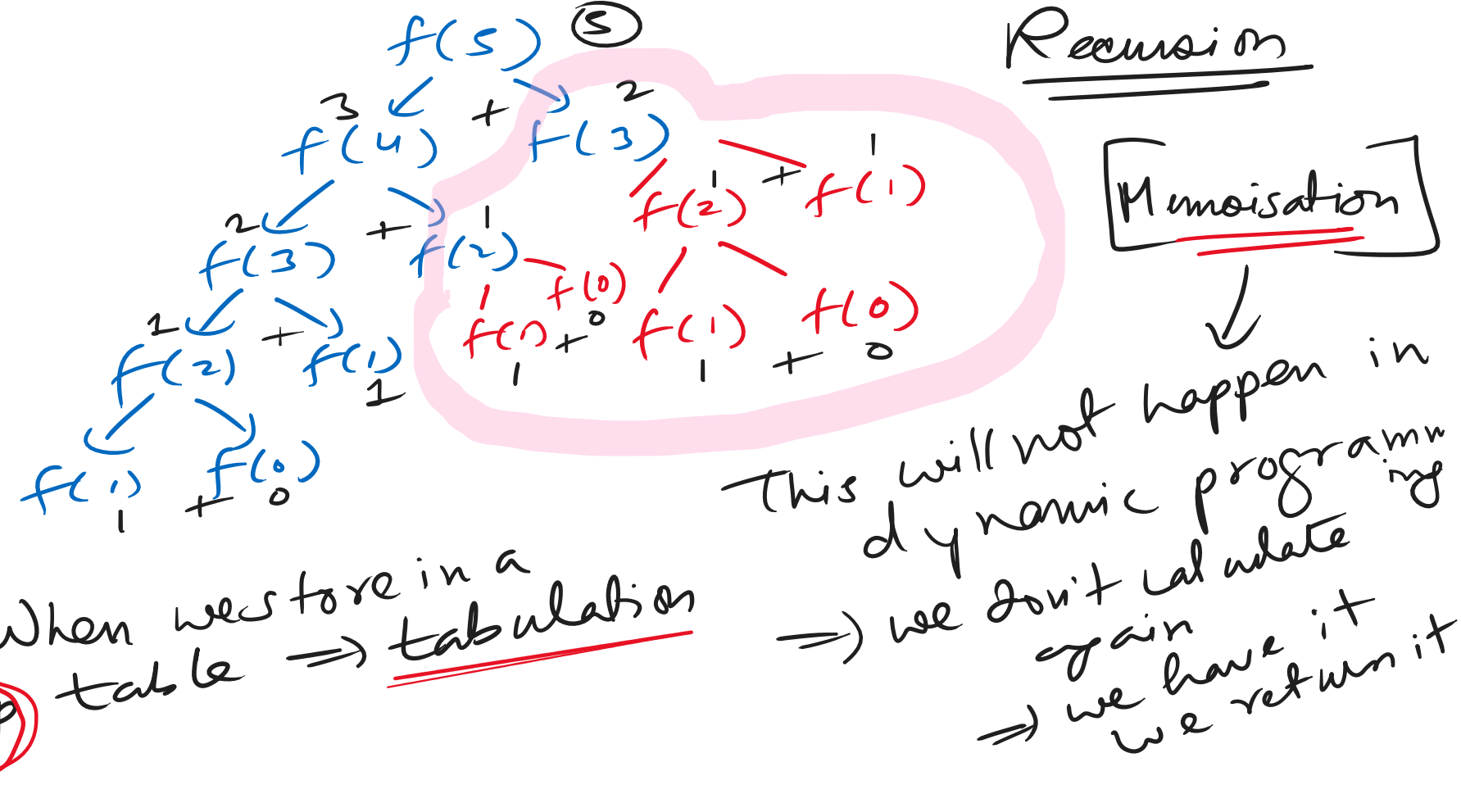
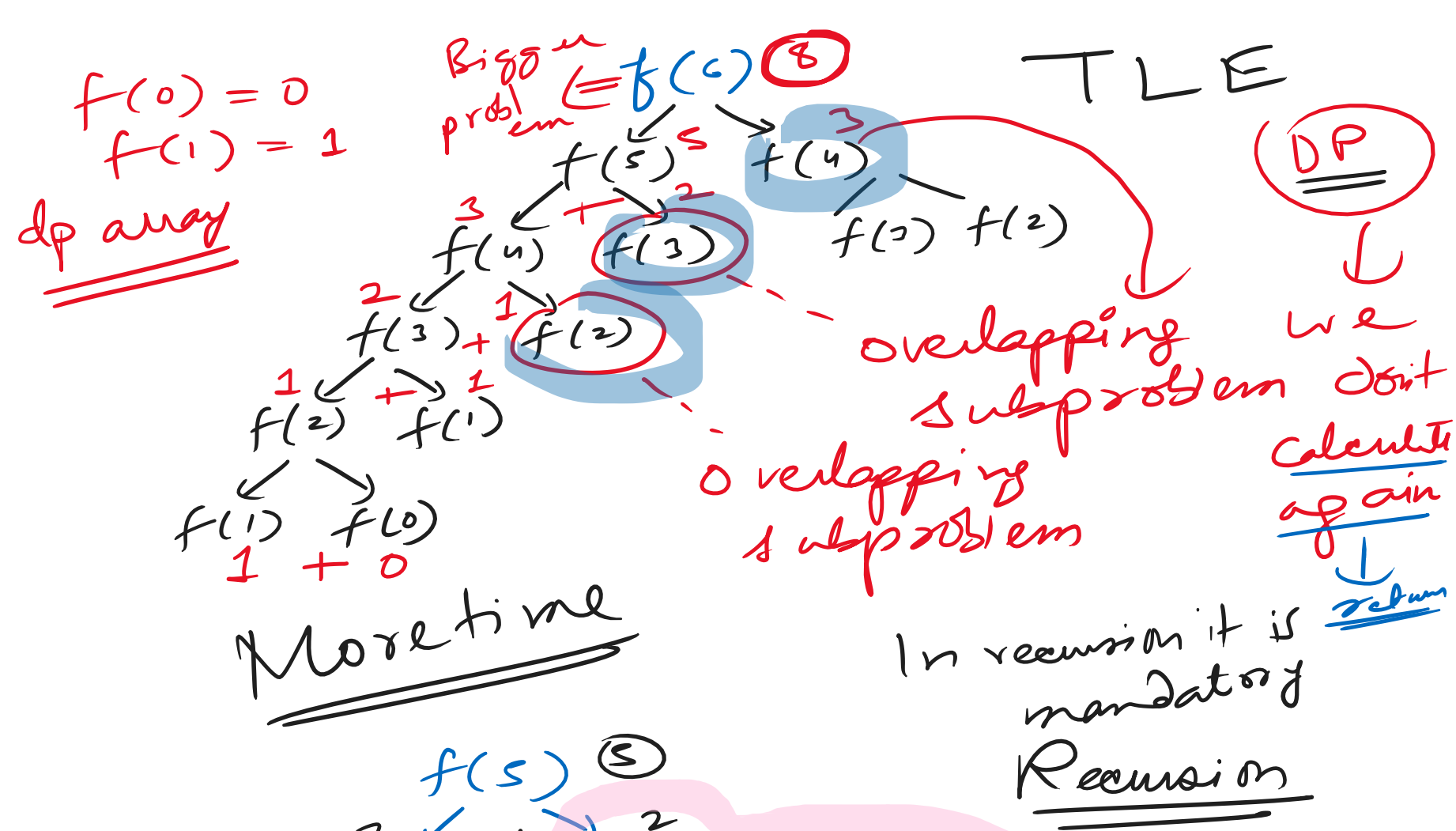
$f(3) = f(2) + f(1)$

$\vdots$

$f(n) = f(n-1) + f(n-2)$

cur = p1 + p2

$n \rightarrow \text{prev}$ (recursion)



- * Recursion $\rightarrow$ Normal Recursion Tree
- * Memorisation $\rightarrow$ When we have a known value, we don't re-calculate, we just return it.
- * Tabulation $\rightarrow$ We store the values (known) in a dp table [1D-dp 2D-dp]
- * Space Optimization $\rightarrow$ If you can optimize the space for a problem, then it's SO. Not possible for all questions/problems. (Completely Eliminate Recursion)

- S1 $\rightarrow$ create a dp array $\rightarrow$ of size $(n+1)$
 - S2 $\rightarrow$ Fill it with -1 values
 - S3 $\rightarrow$ If you have the value, don't calculate just return
 - S4 $\rightarrow$ If you don't have the value then calculate
- $n = 5$
- $\Rightarrow dp = [-1, -1, -1, -1, -1, -1]$
- $n \leq 1 \rightarrow n$
- $dp[5] = dp[4] + dp[3]$

* In Memorisation, the dp array is created in the (main method)

Space complexity = $O(n)$ for a $(n+1)$ size dp array.

Tabulation $\Rightarrow$ Do you have any initial known values? $f(0)$ & $f(1)$

- * Store them in the dp table & make others -1.
- $dp[0] = 0$; $dp[1] = 1$ ($n=6$)

$dp[] = [0, 1, -1, -1, -1, -1, -1]$

initial configuration

Storing known values = Tabulation

Unknown value = (2 to n)

So, we take a for loop from 2 to n & perform the calculation.

(Unknown Values)

- * In Tabulation we generally create the dp array inside the helper function, not in the main method.

* Space Optimisation $\Rightarrow$ Can you solve the question using only variables without any array? Because if we use only variables the Space Complexity becomes $[O(1)] \Rightarrow$ constant

int p2 = 0

int p1 = 1

int cur = p2 + p1

p2 = p1

p1 = cur

$O(1)$ constant

p2 = 0 p1 = 1 cur = 1

1 1 2

1 2 3

2 3 5

3 5 8